

IMC Prosperity 4: Tutorial Round Guide

A Quantitative Trading Primer with Worked Strategies

Prepared for the Prosperity 4 Competition

Spring 2026

Contents

1	Understanding the Competition	2
1.1	What Is Prosperity?	2
1.2	How the Simulation Works	2
2	Core Trading Concepts	2
2.1	The Order Book	2
2.2	Order Matching and Execution	3
2.3	Position and PnL	3
2.4	Position Limits	3
3	Analyzing the Sample Data	4
3.1	Available Data	4
3.2	What to Look For	4
3.3	EMERALDS: The Stationary Asset	4
3.4	TOMATOES: The Trending Asset	5
4	Strategy: Market Making	5
4.1	The Basic Idea	5
4.2	Why It Works Here	6
4.3	Choosing the Half-Spread δ	6
4.4	The Two-Phase Approach	6
5	Fair Value Estimation	6
5.1	EMERALDS: Constant Fair Value	6
5.2	TOMATOES: Exponential Moving Average (EMA)	6
5.2.1	Choosing α	7
5.2.2	Why EMA Over Simple Moving Average?	7
6	Inventory Management	7
6.1	The Problem	7
6.2	Inventory Skew	7
6.2.1	How It Works	8
6.3	Why EMERALDS Doesn't Need Skew	8

7	Code Walkthrough	8
7.1	The <code>run()</code> Method	8
7.2	EMERALDS: Taking Mispriced Orders	9
7.3	EMERALDS: Passive Market Making	9
7.4	TOMATOES: EMA Fair Value	10
7.5	TOMATOES: Inventory Skew	10
8	Common Pitfalls	10
9	Preparing for Future Rounds	11
9.1	What Changes Each Round	11
9.2	Skills to Develop	11
9.3	Useful Mathematical Tools	12
9.4	Practical Workflow for Each Round	12
10	Quick Reference	12
10.1	The <code>Order</code> Class	12
10.2	The <code>run()</code> Return Signature	13
10.3	Key Formulas	13
10.4	Supported Libraries	13

1 Understanding the Competition

1.1 What Is Prosperity?

IMC Prosperity is an algorithmic trading competition where you write a Python program (a `Trader` class) that trades against bots on a simulated exchange. Your goal is to maximize profit (PnL), measured in XIRECs (the in-game currency). Each round introduces new products with different price dynamics, and your algorithm must adapt.

1.2 How the Simulation Works

Your algorithm's `run()` method is called once per **iteration**:

- **Testing**: 1,000 iterations on sample data (instant feedback).
- **Scored run**: 10,000 iterations on unseen data (determines your PnL).

Each iteration, the platform gives you a `TradingState` object containing:

1. The current **order book** for each product (all bot quotes).
2. Your **position** in each product (how many units you hold).
3. Recent **trades** (yours and the market's).
4. A **traderData** string you returned last iteration (for state persistence).

You return a dictionary of `Order` objects. Orders that cross existing bot quotes execute immediately; the rest sit on the book for bots to potentially trade against, then are cancelled if unfilled.

Key Insight

The simulation is **stateless** (runs on AWS Lambda). Class variables and global state may not persist between calls. Always use `traderData` (serialized to a string via `json` or `jsonpickle`) to carry information across iterations.

2 Core Trading Concepts

This section covers the essential market microstructure concepts you need. If you've taken a finance or market microstructure course, skim for Prosperity-specific details.

2.1 The Order Book

An **order book** collects all outstanding buy and sell orders for a product:

Bid (Buy)	Side	Price	Ask (Sell)	Side
14		9,992	—	
29		9,990	—	
—		10,008	14	
—		10,010	29	

Table 1: Typical EMERALDS order book from the sample data.

Key definitions:

- **Best bid** $b^* = \max\{\text{bid prices}\}$: the highest price someone is willing to pay (here, 9,992).
- **Best ask** $a^* = \min\{\text{ask prices}\}$: the lowest price someone is willing to sell at (here, 10,008).
- **Spread** $s = a^* - b^*$: the gap between best ask and best bid (here, 16).
- **Mid-price** $m = \frac{b^* + a^*}{2}$: the midpoint, our simplest fair value estimate (here, 10,000).

2.2 Order Matching and Execution

Orders execute via **price-time priority**:

1. A buy order matches against the *lowest-priced* existing sell order first.
2. A sell order matches against the *highest-priced* existing buy order first.
3. At the same price, the *oldest* order fills first.
4. Execution happens at the **resting order's price** (the order already on the book).

Key Insight

In Prosperity, your orders arrive **instantaneously**—no latency. If you send a buy at 10,008 and there's a sell at 10,008, you will get filled. No bot can “front-run” you.

2.3 Position and PnL

Your **position** in a product is how many units you hold:

$$\text{position}_{t+1} = \text{position}_t + \text{quantity bought} - \text{quantity sold} \quad (1)$$

A positive position is **long** (you own the asset); negative is **short** (you owe the asset). Your PnL at any point is:

$$\text{PnL} = \underbrace{\sum_{\text{sell}} p_{\text{sell}} \cdot q_{\text{sell}} - \sum_{\text{buy}} p_{\text{buy}} \cdot q_{\text{buy}}}_{\text{realized PnL from trades}} + \underbrace{\text{position} \times m_{\text{current}}}_{\text{unrealized PnL from inventory}} \quad (2)$$

where m_{current} is the current mid-price. At simulation end, your final PnL includes the mark-to-market value of any remaining position.

2.4 Position Limits

Each product has a **position limit** L (80 for both tutorial products). This constrains:

$$-L \leq \text{position} \leq L \quad (3)$$

The exchange enforces this *prospectively*: before processing your orders, it checks whether your orders *could* push you past the limit if they all filled:

Warning

If the total buy quantity across all your buy orders for a product would cause $\text{position} + \sum q_{\text{buy}} > L$, **all orders** for that product are rejected. Same for sells: if $\text{position} - \sum q_{\text{sell}} < -L$, all orders rejected. This check applies to buys and sells *independently*.

Therefore, you must cap your order volumes:

$$\text{max_buy} = L - \text{position} \quad (4)$$

$$\text{max_sell} = L + \text{position} \quad (5)$$

Example: If $L = 80$ and $\text{position} = 30$:

- $\text{max_buy} = 80 - 30 = 50$ (buying 50 reaches position +80).
- $\text{max_sell} = 80 + 30 = 110$ (selling 110 reaches position -80).

3 Analyzing the Sample Data

Before writing any code, you should analyze the provided CSV files to understand each product's price dynamics. This is the single most important step.

3.1 Available Data

The tutorial round provides four files:

- `prices_round_0_day_-1.csv` and `day_-2.csv`: Order book snapshots every 100ms (10,000 rows per product per day).
- `trades_round_0_day_-1.csv` and `day_-2.csv`: Every trade that occurred.

3.2 What to Look For

For each product, you want to determine:

1. **Fair value:** What is the “true” price? Is it constant or shifting?
2. **Volatility:** How much does the price move? (standard deviation)
3. **Spread:** How wide is the bid-ask spread? Can you quote inside it?
4. **Autocorrelation:** Do price changes predict future changes? (mean-reverting vs. trending)
5. **Volume:** How often do trades occur? What sizes?

3.3 EMERALDS: The Stationary Asset

Analysis of the sample data reveals:

Metric	Value
Mean mid-price	10,000.00
Std. deviation	0.72
% of time at exactly 10,000	96.7%
Typical spread	16
Lag-1 autocorrelation of Δp	−0.49

Key Insight

EMERALDS is a textbook **stationary, mean-reverting** asset. Fair value is a known constant. The negative autocorrelation (−0.49) means any deviation from 10,000 is quickly corrected. This makes it ideal for a simple market-making strategy.

3.4 TOMATOES: The Trending Asset

Metric	Value
Mean mid-price (Day −2)	5,008
Mean mid-price (Day −1)	4,978
Day-over-day shift	−30
Std. deviation	~20
Typical spread	13–14
Lag-1 autocorrelation of Δp	−0.42

Key Insight

TOMATOES **trends** at longer horizons (30-point shift between days, 15–30 intra-day) but is mean-reverting at short horizons (−0.42 autocorrelation). A static fair value will fail; you need a **dynamic estimate** that adapts to the drift.

4 Strategy: Market Making

Market making is the core strategy for both products. The idea is simple: simultaneously offer to buy slightly below fair value and sell slightly above it, capturing the **spread** as profit.

4.1 The Basic Idea

Suppose fair value is V and you post:

$$\text{Bid (buy) at } V - \delta \quad (6)$$

$$\text{Ask (sell) at } V + \delta \quad (7)$$

where δ is your **half-spread**. If both sides fill, your profit per unit is:

$$\pi = (V + \delta) - (V - \delta) = 2\delta \quad (8)$$

Example: For EMERALDS with $V = 10,000$ and $\delta = 2$:

- Buy at 9,998, sell at 10,002.
- Profit per round trip: $2 \times 2 = 4$ XIRECs per unit.

4.2 Why It Works Here

The bots quote a spread of 16 for EMERALDS (bid 9,992 / ask 10,008). Our spread of 4 offers *strictly better* prices:

- A bot wanting to **sell** gets 9,998 from us vs. 9,992 from other bots: **+6 better**.
- A bot wanting to **buy** pays 10,002 to us vs. 10,008 from other bots: **6 cheaper**.

Due to price-time priority, our orders sit at the top of the book and get filled first.

4.3 Choosing the Half-Spread δ

The optimal δ balances two forces:

$$\underbrace{\text{tighter spread}}_{\delta \downarrow} \implies \begin{cases} \text{More fills (higher volume)} & \uparrow \\ \text{Less profit per fill} & \downarrow \\ \text{More adverse selection risk} & \uparrow \end{cases} \quad (9)$$

Adverse selection: If a bot trades against you, it might be because the price is about to move against your new position. With EMERALDS (near-zero volatility), this risk is negligible, so we can use a tight spread ($\delta = 2$). With TOMATOES (volatile), we use a wider spread ($\delta = 3$) for safety.

4.4 The Two-Phase Approach

Our algorithm uses a two-phase strategy each iteration:

1. **Phase 1 — Take (aggressive orders):** Immediately buy any asks below V and sell to any bids above V . These are guaranteed-profitable trades against mispriced quotes.
2. **Phase 2 — Make (passive orders):** Post resting limit orders at $V \pm \delta$ with whatever position capacity remains. These sit on the book for bots to fill.

We process “take” first because those orders consume volume from the existing book, and we need to track exactly how much position capacity they use before sizing our “make” orders.

5 Fair Value Estimation

5.1 EMERALDS: Constant Fair Value

For EMERALDS, the data overwhelmingly shows $V = 10,000$. No estimation needed.

5.2 TOMATOES: Exponential Moving Average (EMA)

For a trending asset, we need a **dynamic** fair value estimate. The Exponential Moving Average (EMA) is a standard tool:

$$\text{EMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \text{EMA}_{t-1} \quad (10)$$

where:

- x_t is the current observation (mid-price at iteration t).
- $\alpha \in (0, 1)$ is the **smoothing factor**.
- EMA_{t-1} is the previous estimate.

5.2.1 Choosing α

The effective lookback window of an EMA is approximately $\frac{2}{\alpha} - 1$ observations:

α	Effective lookback	Behavior
0.1	~ 19 observations	Very smooth, slow to react
0.3	~ 5.7 observations	Moderate (our choice)
0.5	~ 3 observations	Responsive, noisy
0.9	~ 1.2 observations	Nearly raw price

We use $\alpha = 0.3$ as a balance: responsive enough to track TOMATOES’ intra-day trends (which develop over 50–200 ticks) but smooth enough to filter tick-by-tick noise.

5.2.2 Why EMA Over Simple Moving Average?

1. **Recency bias:** EMA weights recent data exponentially more, which is desirable for a trending asset.
2. **Constant memory:** EMA requires storing only one number (EMA_{t-1}), while SMA requires storing the entire window. This matters because `traderData` is limited to 50,000 characters.
3. **No window artifacts:** SMA can “jump” when old data leaves the window; EMA transitions smoothly.

6 Inventory Management

6.1 The Problem

A pure market maker posts symmetric quotes around fair value. But if the price trends, one side fills more than the other, building up a large position. For example:

- Price rises steadily $\implies$ our sell orders keep filling $\implies$ we go short.
- If the price keeps rising, our short position loses money.

6.2 Inventory Skew

The standard solution is to **skew** our quotes based on our current position. We shift both bid and ask by an amount proportional to our inventory:

$$\text{skew} = -\kappa \cdot \text{position} \tag{11}$$

where $\kappa > 0$ is the skew intensity. Then:

$$\text{bid price} = V - \delta + \text{skew} \tag{12}$$

$$\text{ask price} = V + \delta + \text{skew} \tag{13}$$

6.2.1 How It Works

Position	Skew ($\kappa = 0.05$)	Bid	Ask
+40 (long)	-2	$V - 5$	$V + 1$
0 (flat)	0	$V - 3$	$V + 3$
-40 (short)	+2	$V - 1$	$V + 5$

Table 2: Inventory skew for TOMATOES ($\delta = 3$, $\kappa = 0.05$).

- **Long position:** Skew is negative, shifting quotes *down*. The ask drops closer to fair value, making it easier to sell and unwind the position. The bid moves further away, discouraging more buying.
- **Short position:** Skew is positive, shifting quotes *up*. The bid rises closer to fair value, making it easier to buy and cover. The ask moves further away, discouraging more selling.
- **Flat:** Symmetric quotes, pure spread capture.

This creates a **mean-reverting force** on inventory: the further we deviate from zero, the stronger the pull back.

6.3 Why EMERALDS Doesn't Need Skew

EMERALDS has near-zero volatility. Even at maximum inventory (± 80), the mark-to-market risk is:

$$\text{Risk} = |\text{position}| \times \sigma \approx 80 \times 0.72 \approx 57.6 \text{ XIRECs} \quad (14)$$

This is negligible compared to the spread profit. For TOMATOES with $\sigma \approx 20$:

$$\text{Risk} = 80 \times 20 = 1,600 \text{ XIRECs} \quad (15)$$

This is substantial, so inventory management is essential.

7 Code Walkthrough

This section walks through the key parts of `trader.py`.

7.1 The `run()` Method

Listing 1: Main entry point called each iteration.

```

1 def run(self, state: TradingState):
2     # Restore state from previous iteration
3     trader_data = {}
4     if state.traderData:
5         try:
6             trader_data = json.loads(state.traderData)
7         except (json.JSONDecodeError, TypeError):
8             trader_data = {}
9
10    result = {}

```

```

11     for product in state.order_depths:
12         if product == "EMERALDS":
13             result[product] = self.trade_emeralds(state)
14         elif product == "TOMATOES":
15             result[product], trader_data = self.trade_tomatoes(
16                 state, trader_data
17             )
18
19     traderData = json.dumps(trader_data)
20     conversions = 0
21     return result, conversions, traderData

```

Key points:

- Lines 2–7: Deserialize persistent state. The `try/except` handles the first iteration (empty string) and any corruption.
- Lines 9–15: Dispatch to product-specific strategies. Each returns a list of `Order` objects.
- Line 17: Serialize state back to a string for next iteration.
- Line 19: Return the three required values: orders, conversions (0 for tutorial), and trader data.

7.2 EMERALDS: Taking Mispriced Orders

Listing 2: Phase 1: Buy cheap asks and sell to expensive bids.

```

1  # Buy any sell orders below fair value (cheapest first)
2  if order_depth.sell_orders:
3      for ask_price in sorted(order_depth.sell_orders.keys()):
4          if ask_price < fair_value and buy_vol < max_buy:
5              available = -order_depth.sell_orders[ask_price]
6              qty = min(available, max_buy - buy_vol)
7              if qty > 0:
8                  orders.append(Order(product, ask_price, qty))
9                  buy_vol += qty

```

Key points:

- `sell_orders` values are *negative* (convention), so we negate to get the actual volume (line 5).
- We iterate from lowest to highest ask: grab the cheapest deals first.
- `qty` is capped by both available volume and our remaining position capacity (line 6).
- We accumulate `buy_vol` to ensure total buys never exceed `max_buy` (Eq. 4).

7.3 EMERALDS: Passive Market Making

Listing 3: Phase 2: Post resting orders inside the bot spread.

```

1  mm_bid_price = fair_value - 2    # 9,998
2  mm_ask_price = fair_value + 2    # 10,002

```

```

3
4 remaining_buy = max_buy - buy_vol
5 remaining_sell = max_sell - sell_vol
6
7 if remaining_buy > 0:
8     orders.append(Order(product, mm_bid_price, remaining_buy))
9 if remaining_sell > 0:
10    orders.append(Order(product, mm_ask_price, -remaining_sell))

```

We use all remaining position capacity for market-making orders. The `remaining_*` calculation ensures we never exceed position limits even combining “take” and “make” volumes.

7.4 TOMATOES: EMA Fair Value

Listing 4: Dynamic fair value estimation using EMA.

```

1 mid_price = (best_bid + best_ask) / 2
2
3 alpha = 0.3
4 if ema_key in trader_data:
5     ema = alpha * mid_price + (1 - alpha) * trader_data[ema_key]
6 else:
7     ema = mid_price # Initialize on first iteration
8
9 trader_data[ema_key] = ema
10 fair_value = round(ema)

```

This implements Equation 10. The EMA value is stored in `trader_data` (a Python dict serialized to JSON) so it persists across the stateless iterations.

7.5 TOMATOES: Inventory Skew

Listing 5: Position-based quote adjustment.

```

1 skew = -round(position * 0.05)
2 mm_half_spread = 3
3
4 mm_bid_price = fair_value - mm_half_spread + skew
5 mm_ask_price = fair_value + mm_half_spread + skew

```

With $\kappa = 0.05$ and `position = +40`: `skew = -round(2.0) = -2`, so bid drops to $V - 5$ and ask drops to $V + 1$, aggressively selling to unwind.

8 Common Pitfalls

1. **Exceeding position limits.** If *any* combination of your buy orders could push you past $+L$ (or sell orders past $-L$), the exchange rejects *all* orders for that product. Always compute `max_buy` and `max_sell` per Equations 4–5 and respect them.
2. **Forgetting that sell orders quantities are negative.** The `OrderDepth.sell_orders` dict stores quantities as negative numbers: `{10008: -14}` means 14 units available at 10,008. Always negate before using as a volume.

3. **Not sorting the order book.** The `buy_orders` and `sell_orders` dicts are *not* guaranteed to be sorted. Always use `sorted()` when iterating price levels.
4. **Relying on class variables for state.** The platform runs on AWS Lambda. Class and global variables may reset between iterations. Use `traderData` for anything that must persist.
5. **Timeout.** Your `run()` must return within 900ms (average ≤ 100 ms). Avoid heavy computation. Our algorithm is $O(k)$ where k is the number of price levels—well within limits.
6. **Using a static fair value for TOMATOES.** If you hardcode a fair value for TOMATOES, you will accumulate losing inventory as the price drifts away. Always track the current fair value dynamically.
7. **Submitting without testing.** The simulator gives near-instant feedback. Upload, review the logs, iterate. Check for rejection messages (position limit violations), unexpected PnL, and zero-trade iterations.

9 Preparing for Future Rounds

The tutorial round is a warm-up. Here's what to expect and how to prepare.

9.1 What Changes Each Round

- **New products** with different dynamics (more volatile, correlated, etc.).
- **Manual trading challenges** (inactive in the tutorial, active from Round 1).
- Potentially **conversions** (arbitrage between related products via import/export mechanics).
- **Observations** (external data like sunlight, humidity) that may predict prices.

9.2 Skills to Develop

1. **Data analysis workflow.** For every new product, your first step should be analyzing the CSV data:
 - Plot the mid-price over time. Is it stationary, trending, or mean-reverting?
 - Compute the autocorrelation function. Positive at lag 1 $\implies$ momentum; negative $\implies$ mean reversion.
 - Check for correlations between products.
 - Look at the trade data: who trades with whom, and at what prices?
2. **Fair value models.** Beyond EMA, consider:
 - **VWAP** (Volume-Weighted Average Price) of recent trades.
 - **Linear regression** on observable features (e.g., sunlight $\rightarrow$ price).
 - **Pairs trading**: if two products are correlated, trade the spread between them.
3. **Risk management.** As products get more volatile:
 - Use tighter position limits (self-imposed, below the exchange maximum).

- Widen your spread when volatility increases.
 - Track realized vs. expected PnL to detect strategy breakdown.
4. **Conversion arbitrage.** When products can be “converted” (e.g., buy on one island, transport, sell on another), the profit formula is:

$$\pi_{\text{conversion}} = p_{\text{sell}} - p_{\text{buy}} - \text{transport} - \text{tariffs} \quad (16)$$

Look for cases where this is consistently positive.

5. **Signal extraction from observations.** The `Observation` class may provide data like sunlight or humidity. If you find a statistical relationship between an observation and a product’s price, you can use it as a predictive signal. Test with linear regression:

$$\Delta p_{t+1} = \beta_0 + \beta_1 \cdot \text{observation}_t + \varepsilon_t \quad (17)$$

If β_1 is statistically significant, use the observation to bias your fair value estimate.

9.3 Useful Mathematical Tools

Concept	Application in Prosperity
Exponential Moving Average	Dynamic fair value tracking
Bollinger Bands ($\mu \pm k\sigma$)	Adaptive spread sizing based on volatility
Linear Regression	Predicting price from observations
Correlation / Cointegration	Pairs trading between correlated products
Kelly Criterion	Optimal position sizing given edge and variance
Ornstein–Uhlenbeck Process	Modeling mean-reverting prices

9.4 Practical Workflow for Each Round

1. **Download the data capsule** as soon as the round opens.
2. **Analyze** the new products (plots, statistics, correlations).
3. **Identify** the product type: stationary, trending, correlated pair, observation-driven.
4. **Code** the strategy, starting simple (market-making) and adding complexity only if needed.
5. **Test** on the simulator. Check the logs for position limit rejections, zero-PnL iterations, and unexpected behavior.
6. **Iterate:** tune parameters (α , δ , κ), check PnL breakdown by product, and re-test.
7. **Submit** your final version before the deadline. The *last active upload* is used.

10 Quick Reference

10.1 The Order Class

```

1 Order(symbol: str, price: int, quantity: int)
2 # quantity > 0: BUY order
3 # quantity < 0: SELL order

```

10.2 The run() Return Signature

```

1 def run(self, state: TradingState):
2     ...
3     return result, conversions, traderData
4     # result: Dict[str, List[Order]]
5     # conversions: int (0 if not using)
6     # traderData: str (JSON-serialized state)

```

10.3 Key Formulas

$$\text{Mid-price: } m = \frac{b^* + a^*}{2} \quad (18)$$

$$\text{EMA update: } \text{EMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \text{EMA}_{t-1} \quad (19)$$

$$\text{Max buy volume: } L - \text{position} \quad (20)$$

$$\text{Max sell volume: } L + \text{position} \quad (21)$$

$$\text{Inventory skew: } \text{skew} = -\kappa \cdot \text{position} \quad (22)$$

$$\text{MM bid: } V - \delta + \text{skew} \quad (23)$$

$$\text{MM ask: } V + \delta + \text{skew} \quad (24)$$

$$\text{Spread profit: } \pi = 2\delta \text{ per round trip} \quad (25)$$

10.4 Supported Libraries

All Python 3.12 standard libraries, plus: `pandas`, `numpy`, `statistics`, `math`, `typing`, `jsonpickle`.

Good luck on Intara. May your spreads be tight and your inventory balanced.